Assignment 1: MicroGrad

A Scalar Automatic Differentiation Engine

Joumana Wael
Student ID: 23P0199
Faculty of Engineering, Ain Shams University
CSE485–CSE488: Deep Learning

Abstract

This report describes MicroGrad, a scalar automatic differentiation engine and a small neural-network API built from scratch in Python. Every computation is recorded as a directed acyclic graph (DAG) during the forward pass; each operation stores its own local derivative; and a reverse topological traversal applies the chain rule to obtain the gradient of the output with respect to every node. The engine is used to train a 2–4–1 multilayer perceptron (MLP) on XOR with plain gradient descent, and is verified with an automated suite of 12 unit tests.

1 Introduction

Deep-learning frameworks hide backpropagation behind a single call to `loss.backward()`. The goal of this assignment is to open that box: we implement scalar values, the computational graph, local derivatives, the chain rule, topological sorting and a minimal neural-network API without any deep-learning framework.

The work has two parts: `engine.py` (the `Value` class and backpropagation) and `nn.py` (`Neuron`, `Layer`, `MLP`). The resulting network is trained on XOR, a problem that no single linear neuron can solve, which makes it a meaningful test that gradients flow correctly through hidden layers.

2 The Computational Graph and the `Value` Class

2.1 Graph structure

Each `Value` stores: `data` (the number), `grad` ($\partial L/\partial$ this node, initially 0), `_prev` (the operands it was computed from), `_op` (the operation label) and `_backward` (a closure that pushes the node's gradient to its operands using the operation's local derivative). Applying an operation to `Value` objects creates a new `Value` linked to its operands; these links form a DAG whose leaves are inputs and parameters and whose root is the final output (e.g. the loss).

2.2 Operator overloading

`__add__`, `__mul__` and `__pow__` (numeric exponent only) are implemented as primitives together with `tanh` and `exp`. Negation, subtraction and division are *derived* from them ($-a = a \cdot (-1)$, $a - b = a + (-b)$, $a/b = a \cdot b^{-1}$), and the reflected operators (`__radd__`, `__rmul__`, `__rsub__`, `__rtruediv__`) let plain Python numbers appear on the left. Deriving operations from primitives means they receive correct gradients automatically, with no extra backward code to get wrong.

3 Mathematical Derivations

Let L be the final scalar output, a, b the operands and z the result of an operation. By the chain rule, each operand receives

$$\frac{\partial L}{\partial a} \mathrel{+}= \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial a}. \tag{1}$$

So each operation only needs its *local* derivative $\partial z/\partial a$; the factor $\partial L/\partial z$ is the gradient already stored in `out.grad`.

3.1 Addition

$$z = a + b, \qquad \frac{\partial z}{\partial a} = 1, \quad \frac{\partial z}{\partial b} = 1 \;\Rightarrow\; \frac{\partial L}{\partial a} \mathrel{+}= \frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial b} \mathrel{+}= \frac{\partial L}{\partial z}.$$

Addition simply *distributes* the incoming gradient unchanged to both operands.

3.2 Multiplication

$$z = ab, \qquad \frac{\partial z}{\partial a} = b, \quad \frac{\partial z}{\partial b} = a \;\Rightarrow\; \frac{\partial L}{\partial a} \mathrel{+}= b\,\frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial b} \mathrel{+}= a\,\frac{\partial L}{\partial z}.$$

Each operand's gradient is scaled by the *other* operand's value.

3.3 Power

For a constant exponent n:

$$z = a^n, \qquad \frac{\partial z}{\partial a} = n\,a^{n-1} \;\Rightarrow\; \frac{\partial L}{\partial a} \mathrel{+}= n\,a^{n-1}\frac{\partial L}{\partial z}.$$

The exponent is restricted to a number (not a `Value`) because we only differentiate with respect to the base; $n = 0$ gives $z = 1$ with derivative 0, consistent with the formula.

3.4 Negation, subtraction and division

$$\begin{aligned}
-a = a \cdot (-1) \qquad &\Rightarrow \quad \frac{\partial(-a)}{\partial a} = -1, \\
a - b = a + (-b) \qquad &\Rightarrow \quad \frac{\partial(a-b)}{\partial a} = 1, \quad \frac{\partial(a-b)}{\partial b} = -1, \\
a/b = a \cdot b^{-1} \qquad &\Rightarrow \quad \frac{\partial(a/b)}{\partial a} = \frac{1}{b}, \quad \frac{\partial(a/b)}{\partial b} = a \cdot (-1)\,b^{-2} = -\frac{a}{b^2}.
\end{aligned}$$

The last result follows from the product rule combined with the power rule ($n = -1$). Division is undefined for $b = 0$, so it is rejected with an error.

3.5 Hyperbolic tangent

With $z = \tanh a = \dfrac{e^a - e^{-a}}{e^a + e^{-a}}$, the quotient rule gives

$$\frac{\partial z}{\partial a} = \frac{(e^a + e^{-a})^2 - (e^a - e^{-a})^2}{(e^a + e^{-a})^2} = 1 - \tanh^2 a = 1 - z^2 \;\Rightarrow\; \frac{\partial L}{\partial a} \mathrel{+}= (1 - z^2)\frac{\partial L}{\partial z}.$$

The derivative is expressed through the already-computed output z, so the backward pass needs no extra evaluation of tanh.

3.6 Exponential

$$z = e^a, \qquad \frac{\partial z}{\partial a} = e^a = z \;\Rightarrow\; \frac{\partial L}{\partial a} \mathrel{+}= z\,\frac{\partial L}{\partial z}.$$

3.7 Worked example

Let $L = \tanh(ab + c)$ with $a = 2,\ b = -1,\ c = 0.5$. Forward: $u = ab = -2$, $s = u + c = -1.5$, $L = \tanh(-1.5) \approx -0.9051$. Backward, seeding $\partial L/\partial L = 1$:

$$\begin{aligned}
&\frac{\partial L}{\partial s} = 1 - L^2 \approx 1 - 0.8192 = 0.1808,\\
&\frac{\partial L}{\partial u} = \frac{\partial L}{\partial c} = \frac{\partial L}{\partial s} \approx 0.1808 \quad \text{(addition distributes)},\\
&\frac{\partial L}{\partial a} = b \cdot \frac{\partial L}{\partial u} \approx -0.1808, \qquad \frac{\partial L}{\partial b} = a \cdot \frac{\partial L}{\partial u} \approx 0.3616.
\end{aligned}$$

Each number is a product of local derivatives along the path from L, exactly as the chain rule states.

4 Backpropagation and Topological Sorting

4.1 Chain rule on a graph

If $u = f(a)$ and $L = g(u)$, then $\dfrac{\partial L}{\partial a} = \dfrac{\partial L}{\partial u}\dfrac{\partial u}{\partial a}$. When a node feeds into *several* later operations, its total gradient is the *sum* over all paths (multivariable chain rule). This is why every `_backward` uses `+=` and never `=`.

4.2 Topological ordering

A node may only propagate its gradient once *every* node that depends on it has finished. We obtain a valid order with a depth-first search from the output: mark the node visited, recursively visit its operands (the `_prev` set), and *then* append the node to a list (post-order). In the resulting list every node appears after all of its operands, so iterating over the list *in reverse* processes each node only after all nodes that use it. The backward pass seeds `self.grad` $= 1$ (since $\partial L/\partial L = 1$) and calls `_backward` on each node in this reversed order.

4.3 The two common traps

Post-order trap. If a node were appended *before* visiting its operands (pre-order), it would appear before them in the list; after reversing, its operands would be processed first, using an incomplete `out.grad`, and the chain rule would run out of order.

Diamond-graph trap. For $c = a + a$ the node a is reachable by two edges. Without a `visited` set, the DFS visits a twice (and in larger graphs the repeated visits grow exponentially), so a appears more than once in the list and may be processed more than once. With the set, a appears once; but because `_prev` is a set, the two edges are not duplicated structurally, so correctness comes from the `+=` inside `_backward`: the addition rule runs `self.grad += out.grad` and `other.grad += out.grad` on the same object, giving $\partial c/\partial a = 2$ as expected. The `visited` set and gradient accumulation solve two different problems and both are required.

4.4 Resetting gradients

Because gradients accumulate with `+=`, they persist between calls to `backward()`. Before each training iteration all parameter gradients are set to 0 (`zero_grad`); otherwise the update would use the sum of current and all earlier gradients.

5 Neural Network API

5.1 Neuron

A neuron holds n weights and a bias, each initialised uniformly in $[-1, 1]$:

$$s = \sum_{i=1}^{n} w_i x_i + b, \qquad \hat{y} = \tanh(s).$$

It has $n + 1$ trainable parameters.

5.2 Layer and MLP

A layer is a list of neurons that all receive the same input vector; it returns a list of outputs (or a single `Value` if it has one neuron). An MLP chains layers so that each layer's output is the next layer's input, and exposes `parameters()` and `zero_grad()`.

Layer	Computation	Parameters
Hidden	$2 \to 4$, tanh	$4 \times (2 + 1) = 12$
Output	$4 \to 1$	$1 \times (4 + 1) = 5$
Total		17

Table 1: Parameter count of the 2–4–1 MLP.

6 Training Procedure and Experimental Setup

6.1 Dataset

x_1	x_2	target $x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Table 2: XOR truth table. The classes are not linearly separable, so a hidden nonlinear layer is required.

Python and NumPy seeds are fixed to 42 for reproducibility.

6.2 Loss and optimiser

Mean squared error over $N = 4$ examples, built from `Value` operations so that it is itself a graph node:

$$L = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2.$$

Each iteration: (1) forward pass, (2) compute L, (3) `zero_grad()`, (4) `L.backward()`, (5) gradient-descent update

$$p \leftarrow p - \eta \frac{\partial L}{\partial p}, \qquad \eta = 0.1,$$

repeated for 5000 epochs.

7 Results and Discussion

After training, the network reproduces the XOR table: outputs are essentially 0 for $(0, 0)$ and $(1, 1)$ and about 0.97–0.98 for $(0, 1)$ and $(1, 0)$ (Figure 1). The MSE decreases smoothly and monotonically over all 5000 epochs, reaching roughly 10^{-4} (Figure 2). A smooth monotone curve with no oscillation indicates that the learning rate is appropriate and that the computed gradients point in a consistent descent direction. The curve has not fully flattened at epoch 5000, so more epochs would reduce the loss further.

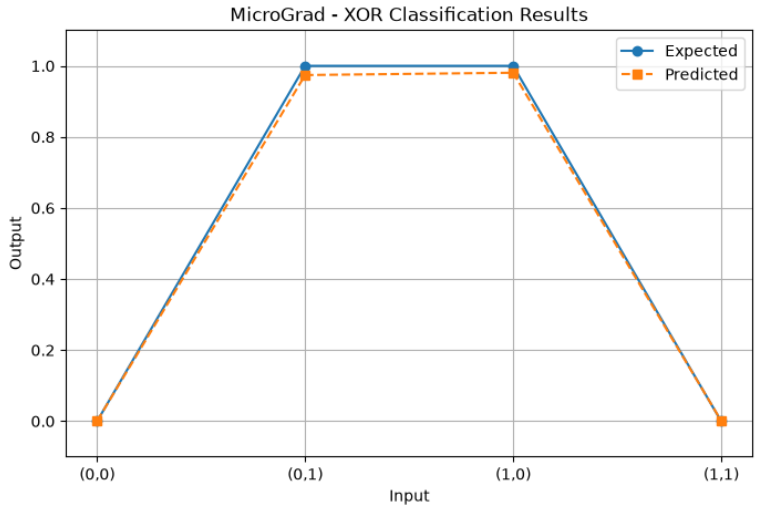


Figure 1: Expected vs. predicted outputs for the four XOR inputs.

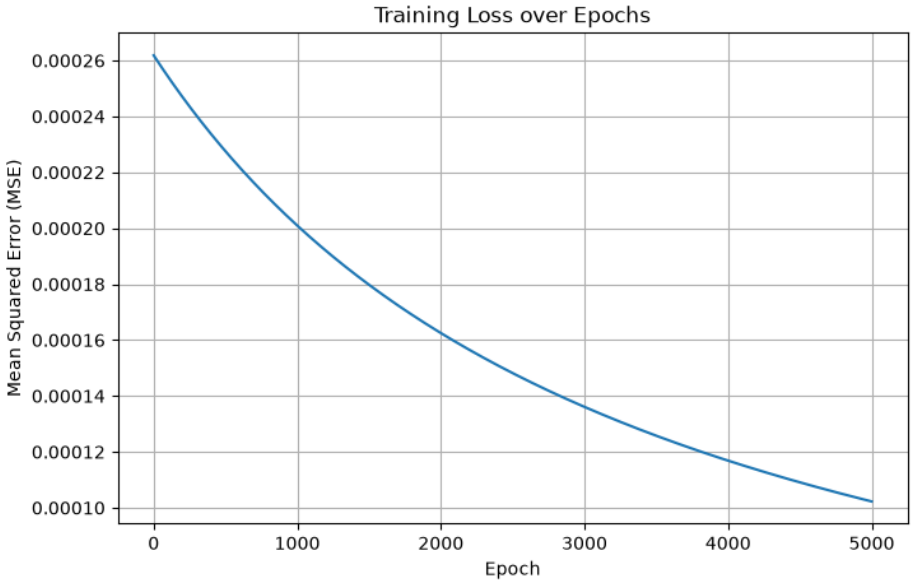


Figure 2: Training MSE over 5000 epochs.

8 Testing and Verification

The provided `pytest` suite (12 tests: 9 for the engine and 3 for the neural-network API, as shown in Figure 3) passes in full. It covers initialisation, basic operations, power, activations, gradient accumulation on a diamond graph, chain-rule behaviour, division by zero, type casting and the neuron/layer/MLP classes.

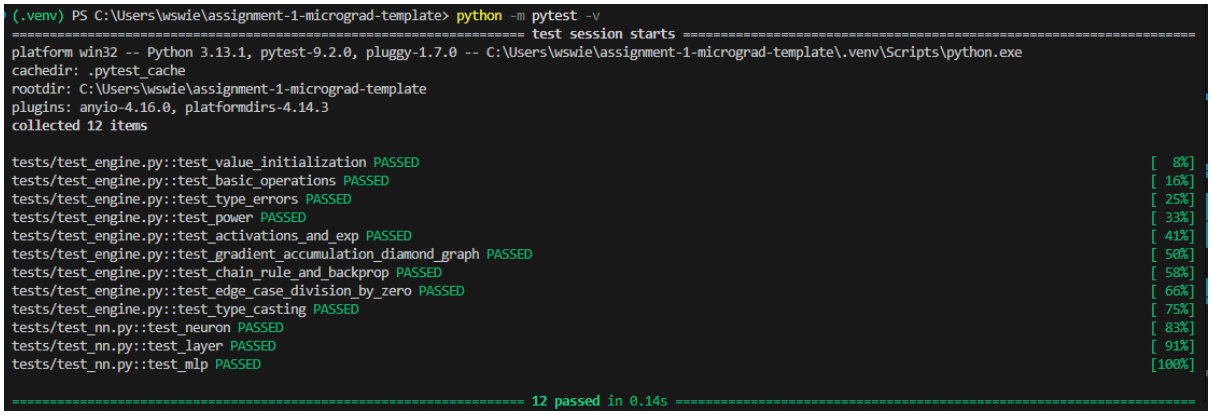


Figure 3: `pytest -v` output: 12 passed.

9 Design Justifications and Conceptual Questions

Why a computational graph? It records which values each result depends on, so the chain rule can be applied systematically to any expression, however it was written.

Why store a local `_backward` per node? Each operation knows only its own local derivative. The global `backward()` stays a single generic traversal, and adding a new operation means writing one small closure.

Why reverse topological order? A node's gradient is a sum over all nodes that use it; it is only complete after all of them have run.

Post-order trap? Appending a node after visiting its operands guarantees operands precede it in the list, which reversed gives a valid backward order.

Diamond-graph trap? A shared node must be traversed once (`visited`) yet receive contributions from every use (`+=`).

Why reset gradients? `+=` accumulates across calls, so stale gradients must be cleared before each update.

Why tanh**?** It is smooth, so the derivative $1 - \tanh^2$ exists everywhere, and it is nonlinear, so stacked layers can represent functions such as XOR; without it the MLP would collapse to a single linear map. It is also zero-centred and bounded in $(-1, 1)$.

Why random initialisation? If all weights started equal, all hidden neurons would receive identical gradients and remain identical (symmetry); random values break this symmetry.

10 Conclusion

We built a scalar autograd engine whose correctness rests on three ideas: local derivatives per operation, accumulation of gradients across all paths, and processing nodes in reverse topological

order. A 2–4–1 MLP built on top of it learns XOR to a loss of about 10^{-4}, confirming that forward and backward passes, gradient accumulation and parameter updates all work together. Possible extensions are more activations (ReLU, sigmoid), additional loss functions and mini-batch training.